

GrillosIA — Esquema de Base de Datos

Este documento describe el esquema relacional usado por GrillosIA para almacenar los datos experimentales (de literatura científica y de ensayos de laboratorio) que alimentan el modelo predictivo de calidad nutricional de harina de grillo.

- **Fuente de verdad del modelo:** [backend/db/models.py](#)
- **Migracion inicial:** [backend/alembic/versions/f6911fb95e73_initial_tables.py](#)
- **Diagrama ER:** [docs/diagrams/er-diagram.svg](#) (PNG, [fuente Mermaid](#))

experiments			
integer	id	PK	autoincrement
timestamp	created_at		default now()
varchar_50	tipo_dieta		NOT NULL – V01 codigo de dieta
float	alimento_g_dia		NOT NULL – V02 g/dia
float	temperatura		NOT NULL – V03 grados C
float	humedad_ambiental		NOT NULL – V04 porcentaje
varchar_100	especie		NOT NULL – V07 especie de grillo
integer	tiempo_desarrollo		NOT NULL – V08 dias
float	fotoperiodo		V05 horas luz/dia
float	densidad		V06 ind/m2
integer	n_grillos_inicio		V09 conteo inicial
float	proteina_harina		V10 TARGET %MS
float	lipidos_harina		V11 TARGET %MS
float	tasa_supervivencia		V12 porcentaje
float	peso_promedio		V13 g/individuo
float	biomasa_total		V14 g total cosechado
text	observaciones		notas libres
varchar_200	fuentes		literatura o experimental + ref

1. Estructura de la tabla `experiments`

Una unica tabla almacena todos los registros experimentales. Cada fila representa un tratamiento (combinacion dieta x condiciones x especie x tiempo) extraido de un articulo cientifico o de un ensayo de laboratorio.

Columnas

#	Columna	Tipo PostgreSQL	Nulo	Codigo catalogo	Rol en el modelo
1	<code>id</code>	<code>integer</code> (autoincrement, PK)	NO	—	Identidad
2	<code>created_at</code>	<code>timestamp</code> (default <code>now()</code>)	si	—	Auditoria
3	<code>tipo_dieta</code>	<code>varchar(50)</code>	NO	V01	Input categorico
4	<code>alimento_g_dia</code>	<code>double precision</code>	NO	V02	Input numerico (g/dia)
5	<code>temperatura</code>	<code>double precision</code>	NO	V03	Input numerico (grados C)
6	<code>humedad_ambiental</code>	<code>double precision</code>	NO	V04	Input numerico (porcentaje)
7	<code>especie</code>	<code>varchar(100)</code>	NO	V07	Input categorico (grillo)
8	<code>tiempo_desarrollo</code>	<code>integer</code>	NO	V08	Input numerico (dias)
9	<code>fotoperiodo</code>	<code>double precision</code>	si	V05	Condicion de cria (h/dia)
10	<code>densidad</code>	<code>double precision</code>	si	V06	Condicion de cria (ind/m2)
11	<code>n_grillos_inicio</code>	<code>integer</code>	si	V09	Registro
12	<code>proteina_harina</code>	<code>double precision</code>	si	V10	TARGET (%MS)
13	<code>lipidos_harina</code>	<code>double precision</code>	si	V11	TARGET (%MS)
14	<code>tasa_supervivencia</code>	<code>double precision</code>	si	V12	Filtro de viabilidad (%)
15	<code>peso_promedio</code>	<code>double precision</code>	si	V13	Filtro de viabilidad (g)
16	<code>biomasa_total</code>	<code>double precision</code>	si	V14	Filtro de viabilidad (g)
17	<code>observaciones</code>	<code>text</code>	si	—	Notas libres (receta, hidratacion)
18	<code>fuente</code>	<code>varchar(200)</code>	si	—	<code>literatura</code> o <code>experimental</code> + ref

Codigos V01–V14 corresponden al catalogo de variables descrito en `.claude/CLAUDE.md` (seccion "Model Variables").

Indices y restricciones

- **PK:** `id` (`experiments_pkey` , `btree`).
- **NOT NULL:** las 6 features que el modelo necesita siempre presentes (`tipo_dieta` , `alimento_g_dia` , `temperatura` , `humedad_ambiental` , `especie` , `tiempo_desarrollo`).
- Sin FKs en esta version: no hay otras tablas a las que apuntar (ver §2).

2. Decisiones de disenio

2.1 Una sola tabla, no esquema normalizado

Se opto por una unica tabla desnormalizada en lugar de un esquema 3FN con tablas separadas para `dietas` , `articulos` , `especies` , `tratamientos` , etc.

Justificacion:

- **Volumen pequeno:** el dataset inicial son ~100–300 filas (literatura). La referencia Vargas-Serna et al. (2025) logro $R^2=0.99$ con 105 puntos de 28 papers. A esta escala, los costos de joins son irrelevantes y la desnormalizacion no introduce problemas de almacenamiento.
- **Workflow del equipo de biologia:** la captura de datos se hace en CSV (`data/literature/template.csv`) con una fila por tratamiento. Un esquema normalizado obligaria a ETL de varios pasos para repartir la fila entre tablas, anadiendo fricciones sin valor real.
- **Consumo del ML:** el entrenamiento (`backend/ml/trainer.py`) carga la tabla completa como un DataFrame de pandas. Una sola tabla = un solo `SELECT *` .
- **Codigos de dieta locales por articulo:** `tipo_dieta` es un codigo (D1, D2, ...) cuyo significado depende del articulo (ver `data/literature/INSTRUCCIONES.md` , §4). No hay una entidad global "Dieta" con composicion estable, asi que crear una tabla `dietas` con FK seria artificial. La receta real va en `observaciones` .
- **Reversible:** si en Mes 7+ el volumen crece o aparecen relaciones reales (ej. multiples mediciones temporales por tratamiento), Alembic permite separar tablas con una migracion incremental.

2.2 Alembic para migraciones

Justificacion:

- **Versionado del esquema:** cada cambio queda registrado como un script Python con `upgrade()` y `downgrade()` . El estado actual lo refleja `alembic_version` .
- **Compatibilidad con SQLAlchemy:** el proyecto ya usa SQLAlchemy 2.x ORM (`db/models.py`). Alembic se integra nativamente y soporta autogenerate (`alembic revision --autogenerate`)

detectando diferencias entre `models.py` y el esquema real.

- **Reproducibilidad:** cualquier desarrollador, CI, o despliegue corre `alembic upgrade head` y obtiene exactamente la misma base de datos.
- **Bajo costo operativo:** una sola tabla, una migracion. No es sobreingenieria porque ya esta listo para cuando se agreguen tablas de `predicciones`, `usuarios`, o `feedback` mas adelante.

2.3 Tipos de datos

Tipo SQLAlchemy	Tipo PostgreSQL	Justificacion
<code>Integer</code> (id, autoincrement)	<code>integer</code> + <code>nextval(...)</code>	PK estandar, suficiente para millones de filas.
<code>DateTime</code> (<code>server_default=now()</code>)	<code>timestamp</code>	Auditoria; el default se calcula del lado del servidor para evitar drift de clock entre clientes.
<code>String(50)</code> para <code>tipo_dieta</code>	<code>varchar(50)</code>	Codigos cortos (D1, D2, ...). 50 caracteres es holgura suficiente.
<code>String(100)</code> para <code>especie</code>	<code>varchar(100)</code>	Nombres binomiales (genero + especie).
<code>String(200)</code> para <code>fuentes</code>	<code>varchar(200)</code>	Citas tipo "Apellido et al. Anio" o nombres de archivos de paper.
<code>Float</code> para todas las medidas continuas	<code>double precision</code>	Mediciones cientificas reportadas con decimales (proteina 62.5%, peso 0.46 g). Precision IEEE 754 doble es mas que suficiente; ningun valor requiere precision arbitraria.
<code>Integer</code> para <code>tiempo_desarrollo</code> , <code>n_grillos_inicio</code>	<code>integer</code>	Conteos discretos.
<code>Text</code> para <code>observaciones</code>	<code>text</code>	Longitud variable, sin limite practico. PostgreSQL <code>text</code> y <code>varchar(n)</code> tienen el mismo rendimiento; usamos <code>text</code> cuando no hay razon para acotar.

Por que `Float` y no `Numeric/Decimal` : las medidas vienen reportadas en papers con 1–2 decimales. No hay requerimiento de exactitud monetaria. `Float` ahorra ~50% de espacio respecto a `Numeric` y los joins/agg son ~3x mas rapidos en PostgreSQL.

Por que sin enum tipado para `especie` o `tipo_dieta` : en este momento las opciones siguen evolucionando (pueden entrar `Gryllodes sigillatus`, `Acheta domesticus`, dietas custom de laboratorio). Mantenerlos como string evita migraciones cada vez que el equipo agrega una opcion. Si en Fase 2 se estabilizan, se promueven a `Enum` con una migracion.

3. Como correr la migracion

Requisitos

- PostgreSQL 14+ corriendo localmente (probado en PostgreSQL 16).
- Python 3.11+ y las dependencias del backend instaladas.

Pasos

```
# 1. Crear la base de datos
createdb grillia

# 2. (Opcional) configurar URL si no se usa el default
# El default es `postgresql://localhost:5432/grillia`. Para overridear,
# exportar la variable de entorno antes de correr alembic:
export GRILLOSIA_DATABASE_URL="postgresql://USUARIO@localhost:5432/grillia"

# 3. Crear entorno virtual e instalar dependencias
cd backend
python3.13 -m venv .venv
.venv/bin/pip install -r requirements.txt

# 4. Aplicar todas las migraciones pendientes
.venv/bin/alembic upgrade head
```

Verificacion

```
psql -d grillia -c "\dt" # debe listar 'experiments' y 'alembic_'
psql -d grillia -c "\d experiments" # estructura completa
psql -d grillia -c "SELECT * FROM alembic_version" # debe mostrar f6911fb95e73
```

Rollback

```
.venv/bin/alembic downgrade base # elimina la tabla experiments
```

4. Carga de datos

El esquema esta listo para ser cargado desde los CSVs que produzca el equipo de biologia siguiendo [data/literature/INSTRUCCIONES.md](#) con base en la plantilla

[data/literature/template.csv](#). Un script de carga (`backend/scripts/load_csv.py`) se entregara como parte del avance del Mes 4 (3.2 al 50%).